

Experiencia Educativa: Aprendizaje Máquina

Dr. David Martínez Galicia

Ingeniería en Ciberseguridad e Infraestructura de Cómputo
Facultad de Economía e Informática
Universidad Veracruzana



Presentación

- Ingeniero en Mecatrónica.
- Maestro en Inteligencia Artificial.
- Doctor en Inteligencia Artificial.



Información del curso

- **Objetivo de la experiencia:** Brindar los fundamentos para analizar grandes **conjuntos de datos**, identificar **anomalías** y **valores atípicos**, así como identificar rápidamente **tendencias** y **patrones**, con el **entramiento** de algoritmos de **aprendizaje automático**, en diferentes ámbitos asociados a la ciberseguridad.
- Miércoles (07:00 – 08:59) y jueves (07:00 – 08:59).
- Oportunidades de evaluación: ordinario, extraordinario y título (si aplica).
- Contacto: davidmartinez02@uv.mx



Criterios de evaluación

Criterio	Porcentaje
Exámenes	40%
Tareas	30%
Proyecto	30%
Total	100%

- Para la acreditación tanto en ordinario, extraordinario y título de suficiencia es necesario obtener un mínimo de 6 en todos los criterios de evaluación.
- Cumplir con los porcentajes de asistencia que indica el estatuto.



Saberes teóricos

Introducción	ST-01. Manejo de datos	ST-02. Aprendizaje supervisado
- Inteligencia Artificial (IA) - Definición de la IA - Prueba de Turing - Campos de la IA	- Estructurados y no estructurados - Etiquetados y no etiquetados - Inconsistencia de datos	- Redes neuronales - Redes bayesianas - Árboles de decisión
ST-03. Aprendizaje no supervisado	ST-04. Aprendizaje por refuerzo	ST-05. Evaluación de modelos
- K-means - Variantes de K-means	- Q-learning	- Precisión - Sensibilidad y especificidad - ROC - Validación cruzada - Distancia intra e inter clúster



Calendarización

Mes	Febrero			Marzo				Abril					Mayo				Jun.	
Semana	2°	3°	4°	1°	2°	3°	4°	1°	2°	3°	4°	5°	1°	2°	3°	4°	1°	
Introducción								SS										
ST-01																		
ST-02						18												
ST-03																		
ST-04																		
ST-05																		
Notación: SS = Semana Santa. Días de suspensión en blanco.																		



Nacimiento de la Inteligencia Artificial

- El concepto de Inteligencia Artificial (IA) se acuña en 1956.
- Surge como un campo de estudio en un taller en Dartmouth College.
 - *Cada aspecto de la inteligencia puede ser descrito con suficiente precisión que puede fabricarse una máquina para simularlo.*
- Muchas preguntas, pocas respuestas y aún más esperanzas.



Objeto de estudio de la Inteligencia Artificial

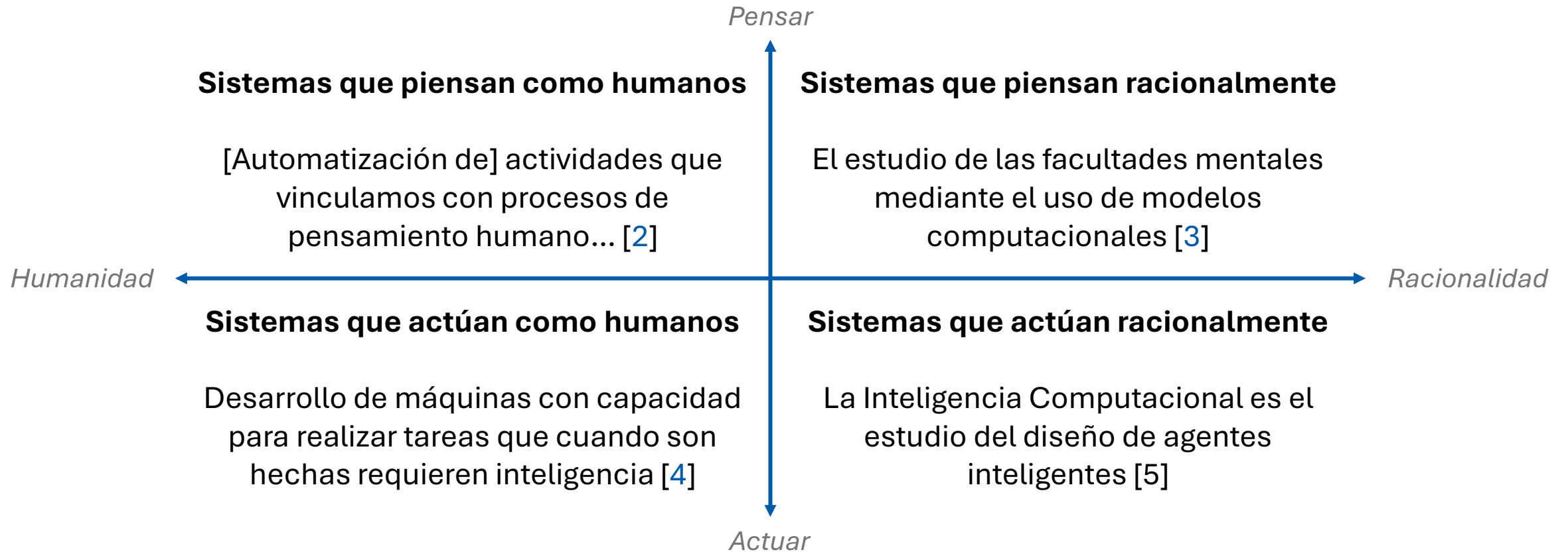
- Rusell y Norvig [1] analizaron ocho libros de texto.
- Encontraron cuatro enfoques sobre cuál debería ser el objeto de estudio.
- Concuerdan en que la IA debe de estudiar sistemas inteligentes.
- ¿Qué características deben tener para ser inteligentes?

Cognición	Comportamiento
Pensar (crear)	Humano
Actuar (simular)	Racional

[1] Russell, S. J., y Norvig, P. (2021). *Inteligencia artificial: un enfoque moderno* (4.^a ed.). Pearson.



Definición de la Inteligencia Artificial



[2] Bellman, R. E. (1978). *An introduction to artificial intelligence: Can computers think?* Boyd & Fraser.

[3] Charniak, E., & McDermott, D. (1985). *Introduction to artificial intelligence*. Addison-Wesley.

[4] Kurzweil, R. (1990). *The age of intelligent machines*. MIT Press.

[5] Poole, D. L., et al. (1998). *Computational intelligence: A logical approach*. Oxford University Press.



Simulación del comportamiento humano

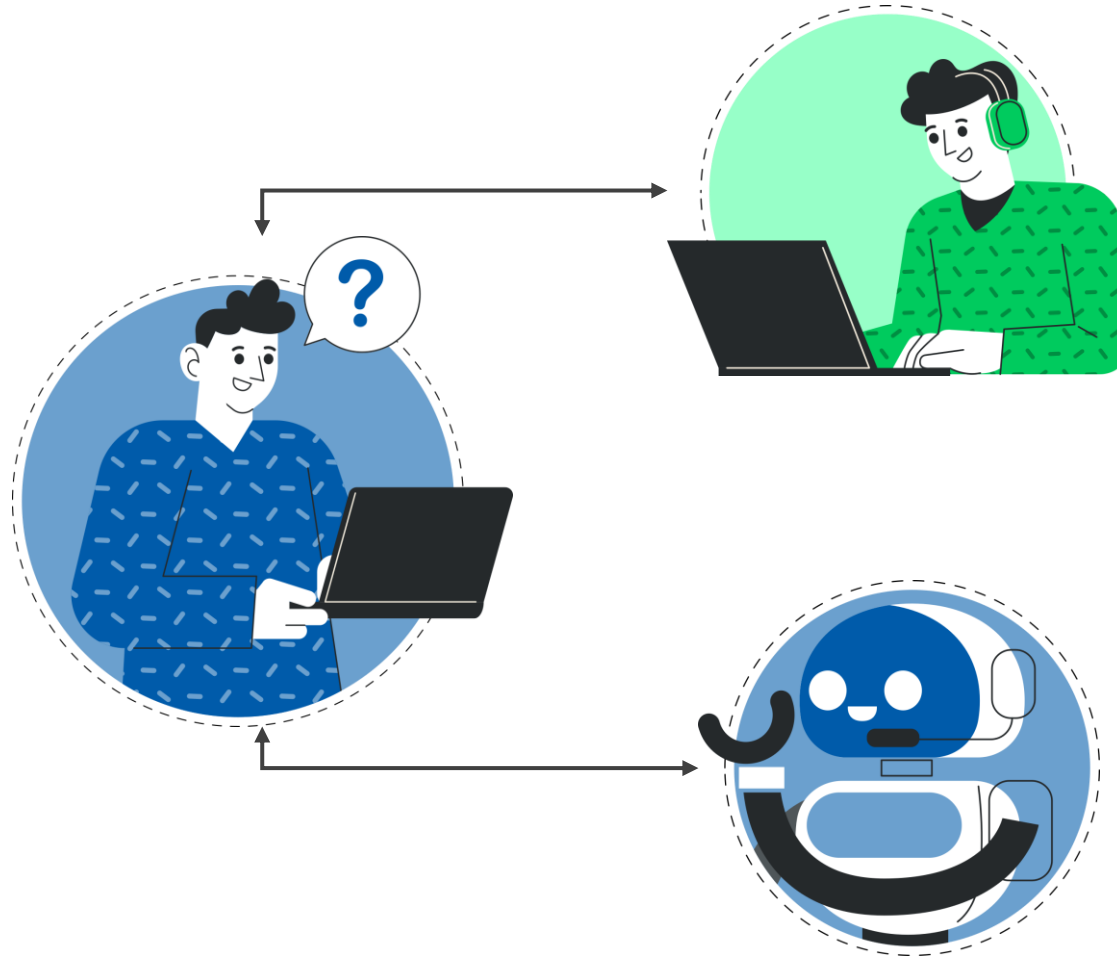
- La Prueba de Turing [6] fue propuesta por Alan Turing (1950).
- Se diseñó para proporcionar una definición de inteligencia.
- No proporciona una lista de cualidades para obtener inteligencia.
- Se basa en la incapacidad de diferenciar entre entidades inteligentes.
- La prueba se supera si no se distingue entre una persona y un sistema.

[6] Turing, A. M. (1950). *Computing machinery and intelligence*. Mind, 59(236), 433–460.



La prueba de Turing

Habitación 1:
Evaluador humano
Cuenta con dos terminales de texto para comunicarse.



Habitación 2:
Ser humano
Recibe preguntas y responde por texto.

Habitación 3:
Sistema de IA
Recibe las mismas preguntas y responde por texto.



Capacidades de la prueba de Turing

El sistema debería poseer las siguientes capacidades:

- **Procesamiento de lenguaje natural** que le permita comunicarse en inglés.
- **Representación del conocimiento** para almacenar lo que se conoce.
- **Razonamiento automático** para responder preguntas usando información.
- **Aprendizaje máquina** para adaptarse a circunstancias y problemas.



La prueba global de Turing

- La prueba original evitó la interacción física entre evaluador y sistema.
- La prueba global [7] también evalúa la percepción y la manipulación física.
- Para superar la prueba global de Turing la computadora debe incluir:
 - **Visión computacional** para percibir objetos.
 - **Robótica** para manipular y mover objetos.
- Estas seis capacidades constituyen los campos de la IA actual.

[7] Harnad, S. (1989). *Minds, machines and Searle*. Journal of Experimental & Theoretical Artificial Intelligence, 1(1), 5–25.



Aprendizaje máquina

- El aprendizaje se refiere a un espectro de situaciones en las cuales un agente incrementa su conocimiento para cumplir una tarea.
- Según Tom Mitchell [8], un programa de computadora aprende de la **experiencia E**, con respecto a alguna clase de **tareas T** y una **medida de desempeño P**, si su desempeño en las tareas de T, medido por P, mejora con la experiencia E.

[8] Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill.



Tipos de aprendizaje

- Según el tipo de experiencia, se distinguen tres tipos de aprendizaje:
 - El **aprendizaje supervisado** consiste en aprender una función a partir de ejemplos de sus entradas y sus salidas.
 - El **aprendizaje no supervisado** consiste en aprender a partir de patrones de entradas que no tienen valores de salidas.
 - El **aprendizaje por refuerzo** consiste en aprender una forma de actuar usando un refuerzo (recompensa o penalización).



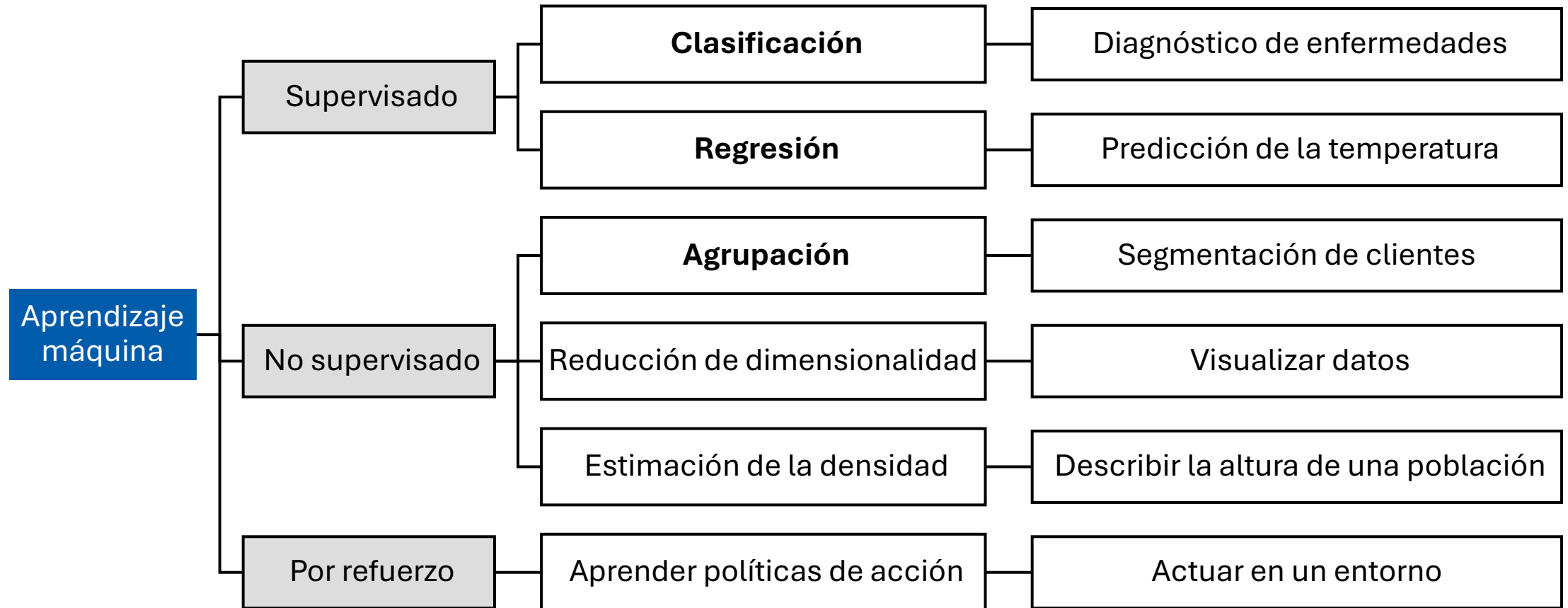
Ejemplos de aprendizaje

- Un sistema puede usar varios tipos de aprendizaje para distintas tareas.

SISTEMA TAXISTA		
Supervisado	No supervisado	Por refuerzo
Cuando el instructor de conducción grite “frena”, el agente puede aprender una regla condición-acción sobre cuándo frenar.	Desarrollar gradualmente los conceptos “día de tráfico bueno” y “día de tráfico malo”, sin que se le haya dado una solución.	La falta de propina al final del viaje da al agente algunas indicaciones de que su comportamiento no es el deseable.



Tareas de aprendizaje



Introducción al manejo de datos

- La calidad de los datos determina la calidad del modelo.
- Sin una preparación adecuada de los datos, no hay aprendizaje efectivo
- Principio fundamental: "Garbage in, garbage out".
- Transformamos datos crudos en patrones para la toma de decisiones.
- Datos limpios detectan amenazas reales en ciberseguridad.
- Se clasifican los datos por su estructura, etiquetas y consistencia.



Datos estructurados

- Datos organizados en formato tabular con esquema definido.

Características:

- Formato predecible (filas y columnas).
- Tipos de datos definidos (enteros, flotantes, categóricos).
- Fáciles de almacenar y consultar en bases de datos relacionales.

Ejemplos:

- Tablas SQL.
- Hojas de cálculo Excel/CSV.
- Logs de sistemas con formato fijo.



Datos no estructurados

- Datos sin formato predefinido ni organización tabular.

Características:

- Sin esquema fijo.
- Requieren procesamiento previo para extraer información.
- Representan ~80% de los datos empresariales.

Ejemplos:

- Texto (correos, código fuente, documentos, páginas web).
- Imágenes y videos.
- Audio y grabaciones.



Datos etiquetados

- Conjuntos de datos donde cada ejemplo tiene una etiqueta/clase asociada.

Características:

- Requieren proceso manual o semi-automático de etiquetado.
- Esenciales para entrenar modelos supervisados.
- Costosos de obtener y validar.

Ejemplos:

- Correos etiquetados como "spam" o "no spam".
- Imágenes de rostros con nombre de persona.
- Transacciones marcadas como "fraudulentas" o "legítimas".



Datos no etiquetados

- Datos sin categorías o respuestas conocidas.

Características:

- Abundantes y fáciles de recolectar.
- Requieren técnicas de descubrimiento de patrones.
- Útiles para detección de anomalías.

Ejemplos:

- Tráfico de red.
- Comportamientos de usuario.
- Código malicioso.



Repositorio de datos

- Los repositorios son bancos de datos públicos y organizados.
- Proporcionan conjuntos de datos para entrenar y validar modelos.
- Ofrecen datos etiquetados y documentados para distintos problemas.
- Son esenciales para la investigación y el desarrollo en aprendizaje máquina.
- Ejemplos: Kaggle, UCI, OpenML, gubernamentales y académicos.
- Su uso acelera el desarrollo y facilita la reproducibilidad.



Formatos de datos de texto

- Los formatos de texto permiten almacenar y procesar datos eficientemente.
- Son los más comunes para cargar en la mayoría de las herramientas.
 - CSV: formato tabular simple y universalmente compatible.
 - TSV: similar al CSV, pero con tabulaciones como delimitador.
 - JSON: ligero y flexible para datos estructurados y anidados.
 - XML: formato estructurado y autodescriptivo, aunque verboso.
 - ARFF: incluye metadatos, usado comúnmente en Weka.
 - Texto plano: sin formato, para datos sin estructura o documentos.



Limpieza y preprocesamiento de datos

- Los datos del mundo real son inexactos, inconsistentes e incompletos.
- El preprocesamiento de datos asegura la integridad de los datos.
- Permite generar análisis confiables y conclusiones válidas.
- Evita decisiones incorrectas o modelos ineficaces.



Flujo de trabajo de limpieza y preprocesamiento

- Recopilación: Obtener datos de bases de datos, APIs o scraping.
- Limpieza: Corregir errores, eliminar duplicados y manejar valores faltantes.
- Integración: Unir datos de múltiples fuentes, resolviendo inconsistencias.
- Transformación: Codificar, normalizar y crear nuevas variables.
- Reducción: Seleccionar características para simplificar el análisis.



Ejemplo de un flujo de trabajo

Recopilación	Limpieza	Integración	Transformación	Reducción
Fuentes comunes: Bases de datos, APIs, archivos CSV, scraping web.	Eliminar duplicados y observaciones irrelevantes. Manejar valores faltantes: Eliminar, imputar o marcar.	Alinear columnas y resolver conflictos entre fuentes.	Codificar variables categóricas ("Sí/No" a 1/0) Normalizar características numéricas (escalar entre 0 y 1)	Eliminar variables irrelevantes. Reducir dimensionalidad combinando variables.
Datos de ventas de múltiples tiendas.	Corregir errores tipográficos en nombres de productos.	Unir datos de clientes y ventas.	Crear características derivadas (edad a partir de fecha de nacimiento).	Usar solo 10 de 50 características para un modelo predictivo.



Python para el aprendizaje máquina

- **Python:** Lenguaje preferido por su facilidad y bibliotecas especializadas.
- Bibliotecas importantes:
 - Pandas: Manipulación de datos.
 - NumPy: Cálculos numéricos.
 - SciPy: Cómputo científico, estadística y señales.
 - Matplotlib/Seaborn: Visualización de datos.
 - Scikit-learn: Machine learning y preprocesamiento.



Pandas para la manipulación de datos

- Funcionalidades :
 - Filtrado, ordenamiento, agregación y fusión de datos.
 - Lectura y escritura de archivos (CSV, Excel, etc.).

```
# Uso de datos desde un archivo
import pandas as pd
df = pd.read_csv('datos.csv')
df.head()
```

```
# Uso de datos precargados
import pandas as pd
import seaborn as sns
df = sns.load_dataset('iris')
df.head()
```

- Uso típico: Limpieza, transformación y análisis exploratorio



NumPy para cálculos numéricos

- Funcionalidades:
 - Arreglos multidimensionales y operaciones matemáticas eficientes.
 - Base para muchas operaciones en Pandas y Scikit-learn.

```
import numpy as np                                # Carga de la biblioteca
arr = np.array([1, 2, 3, 4, 5])                   # Definición de arreglos
arr2 = arr * 2                                     # Multiplicación vectorial
```

- Uso típico: Cálculos numéricos y transformaciones de datos.



SciPy para cómputo científico y estadístico

- Contiene una amplia colección de algoritmos y funciones:
 - stats: pruebas estadísticas, distribuciones, medidas descriptivas.
 - signal: procesamiento de señales.
 - optimize: optimización y ajuste de curvas.
 - interpolate: interpolación de datos.
 - linalg: álgebra lineal avanzada.
- Se integra con NumPy (estructuras de datos) y Pandas (análisis de datos).



Matplotlib para la visualización de datos

- Funcionalidades:
 - Creación de gráficos (líneas, barras, dispersión, etc.).
 - Diagnóstico de problemas de calidad de datos.

```
import matplotlib.pyplot as plt          # Carga de la biblioteca
plt.plot([1, 2, 3, 4, 5])                # Creación de una gráfica
plt.title('Gráfico de línea simple')     # Título de la gráfica
plt.show()                               # Visualización de la gráfica
```

- Uso típico: Análisis exploratorio y visualización de datos.



Seaborn para la visualización estadística

- Funcionalidades:
 - Gráficos atractivos e informativos con pocas líneas de código.
 - Integración con Pandas para análisis rápido.

```
import seaborn as sns                                # Carga de la biblioteca
df = sns.load_dataset('tips')                        # Uso de datos precargados
sns.histplot(df['total_bill'])                       # Creación de un histograma
plt.title('Histograma de Total Bill')                # Título de la gráfica
plt.show()                                           # Visualización de la gráfica
```

- Uso típico: Visualización avanzada y análisis estadístico.



Caso de estudio

- Un conjunto de datos ampliamente conocido y utilizado para practicar la limpieza y preprocesamiento de datos es el [Titanic Dataset](#).
- Este conjunto contiene información sobre los pasajeros del Titanic, como edad, género, clase del boleto, tarifa, puerto de embarque y si sobrevivieron.

```
import pandas as pd
import seaborn as sns

df = sns.load_dataset('titanic')      # Cargar Titanic desde Seaborn
df.head()                            # Mostrar las primeras 5 filas
```



Análisis exploratorio de datos

Exploratory Data Analysis (EDA)

- El análisis exploratorio es el primer acercamiento sistemático a los datos.
- Su objetivo es comprender la estructura y detectar problemas.
- Permite identificar patrones, tendencias y relaciones entre variables.
- Orienta las decisiones sobre limpieza, transformación y modelado.
- Un EDA sólido evita conclusiones erróneas y modelos mal especificados.
- Se apoya en estadísticas descriptivas y visualizaciones.



Dimensiones y tipos de datos I

- Es importante conocer la forma del conjunto: número de filas y columnas.
- Identificar el tipo de cada variable (numérica, categórica, texto, fecha).
- Verificar que los tipos sean adecuados para el análisis.

```
# Dimensiones del conjunto de datos
df.shape
print('Filas:', df.shape[0], 'Columnas:', df.shape[1])

# Tipos de datos en cada variable
print(df.dtypes)

# Resumen de información
df.info()
```



Tipos e información del conjunto

```
print(df.dtypes)
```

```
survived      int64
pclass        int64
sex           object
age           float64
sibsp         int64
parch         int64
...
adult_male    bool
deck          category
embark_town    object
alive         object
alone         bool
dtype: object
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column      Non-Null Count  Dtype
---  ---
 0   survived    891 non-null    int64
 1   pclass      891 non-null    int64
...
13  alive       891 non-null    object
14  alone       891 non-null    bool
dtypes: bool(2), category(2),
float64(2), int64(4), object(5)
memory usage: 80.7+ KB
```



Estadísticos descriptivos básicos

- Resumen de variables categóricas: frecuencias y proporciones.
- Para variables continuas: media, mediana, mínimo, máximo, cuartiles.
- Permite detectar rangos anómalos y escala de las variables.

```
# Estadísticos para variables numéricas  
df.describe()
```

```
# Estadísticos para variables categóricas  
df['sex'].value_counts()  
df['embarked'].value_counts(dropna=False)
```



Distribuciones y frecuencias

- Analizar cómo se distribuyen los valores de una variable.
- Histogramas (*histograms*) para variables continuas.
- Diagramas de barras (*bar plots*) para variables categóricas.

```
# Histograma de edad
```

```
sns.histplot(df['age'].dropna(), bins=30, kde=True)  
plt.title('Distribución de edad')  
plt.show()
```

```
# Frecuencia de clase de pasajero
```

```
sns.countplot(x='class', data=df)  
plt.title('Pasajeros por clase')  
plt.show()
```



Correlaciones entre variables

- Mide la relación lineal entre dos variables numéricas.
- Tiene valores entre -1 y 1. La cercanía a ± 1 indica fuerte asociación.
- Útil para detectar redundancias y relaciones relevantes.
- No implica causalidad.

```
# Matriz de correlación
corr = df[['age', 'fare', 'pclass', 'sibsp', 'parch']].corr()
print(corr)

# Mapa de calor
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title('Correlaciones numéricas')
plt.show()
```



Visualización inicial de datos

- Gráficos simples que resumen múltiples aspectos de los datos.
- Diagramas de dispersión (*scatter plots*) para pares de variables.
- Matriz de dispersión (*pair plots*) para una vista general multivariada.
- Diagramas de caja (*box plots*) para comparar distribuciones entre categorías.

```
# Dispersión: edad vs tarifa coloreada por supervivencia
```

```
sns.scatterplot(x='age', y='fare', hue='survived', data=df)  
plt.title('Relación edad-tarifa según supervivencia')  
plt.show()
```

```
# Pairplot (solo algunas columnas)
```

```
sns.pairplot(df[['age', 'fare', 'survived']].dropna(), hue='survived')  
plt.show()
```



Identificación de valores faltantes

- Identificar valores faltantes puede ser complejo en datos reales.
- Los datos faltantes pueden tomar varias formas.
- Celdas vacías, marcadores como “N/A” o “-999” o datos mal ingresados.

```
df.isnull()                                # Presencia de valores faltantes
faltantes = df.isnull().sum()              # Conteo de valores faltantes
faltantes

df['age'].isnull()                          # Valores faltantes por columna
df[df['age'].isnull()]                     # Registros con valores faltantes
df[df['deck'].isnull()]
```



Identificación de valores atípicos I

- También son conocidos por el anglicismo *outliers*.
- Valores inusuales que difieren significativamente del resto de los datos.
- **Causas:** errores de medición, errores de procesamiento y anomalías reales.

```
# Carga de librerías
from scipy import stats
import numpy as np

# Cálculo de puntajes Z, cuidado con los valores faltantes
z_scores = np.abs(stats.zscore(df2['fare']))
df2[z_scores > 3]
```



Identificación de valores atípicos II

- **IQR:** Detecta valores fuera de 1.5 veces el rango intercuartil ($Q1 - Q3$).

```
# Carga de librerías
```

```
import numpy as np
```

```
# Cálculo de cuartiles y el rango
```

```
Q1 = np.percentile(df['fare'], 25)
```

```
Q3 = np.percentile(df['fare'], 75)
```

```
IQR = Q3 - Q1
```

```
# Detección
```

```
df[(df['fare'] < (Q1 - 1.5 * IQR)) | (df['fare'] > (Q3 + 1.5 * IQR))]
```



Identificación de valores atípicos III

- Visualización de valores atípicos con gráficos:

```
# Carga de librerías
import matplotlib.pyplot as plt

# Gráfico de caja
plt.boxplot(df['fare'])
plt.show()

# Gráfico de dispersión
plt.scatter(range(len(df['fare'])), df['fare'])
plt.show()
```



Tarea 1: Ejercicio de análisis exploratorio de datos

- Selecciona datos de [seaborn](#) con variables numéricas y categóricas.
- Realiza las siguientes actividades de análisis exploratorio:
 - Carga el conjunto de datos y muestra las primeras 5 filas.
 - Identifica las dimensiones y los tipos de datos de cada variable.
 - Calcula estadísticos descriptivos básicos.
 - Visualiza la distribución de al menos dos variables numéricas.
 - Visualiza la frecuencia de al menos dos variables categóricas.
 - Identifica valores faltantes (si los hay) y calcula su porcentaje.
 - Detecta valores atípicos en una variable numérica usando [boxplot](#) o IQR.
 - Calcula la matriz de correlación y muéstrala con un [heatmap](#).
 - Genera un [pairplot](#) con cuatro variables y colorea por una categórica.
- Entrega una libreta con el código, las visualizaciones y tus observaciones.



Limpieza de datos

Data Cleaning (DC)

- Los datos rara vez vienen limpios y listos para usar.
 - **Valores faltantes:** Datos incompletos que pueden sesgar análisis.
 - **Outliers:** Valores atípicos que distorsionan resultados.
 - **Formato inconsistente:** Datos mal estructurados o inconsistentes.
 - **Duplicados:** Registros repetidos que hacen crecer el conjunto de datos.
 - **Datos redundantes:** Información que no aporta valor.
- Estos problemas pueden sesgar el análisis y generar modelos erróneos.



Preservación de datos originales

- Las técnicas para manejar datos faltantes modifican el conjunto de datos.
- Si el resultado no es satisfactorio, no se puede volver al punto de partida.
- La solución es crear una copia de trabajo antes de cualquier manipulación.

```
# Copia del conjunto original  
df2 = df.copy(deep = True)
```



Técnicas para manejar valores faltantes I

- **Eliminación de registros:** Se usa solo si los valores faltantes constituyen una pequeña fracción del conjunto de datos.

```
# Eliminación de datos faltantes  
df2.dropna(inplace=True)  
# Comprobación de datos faltantes  
df2.isnull().sum( )
```

- **Desventaja:** Puede llevar a la pérdida de información valiosa.



Técnicas para manejar valores faltantes II

- **Sustituir datos faltantes con valores estadísticos:**

- Media – Para datos continuos.
- Mediana – Para datos ordinales.
- Moda – Para datos categóricos.

```
# Imputación de datos
```

```
df2.fillna({'age': df2['age'].mean()}, inplace = True)  
df2.fillna({'age': df2['age'].median()}, inplace = True)  
df2.fillna({'age': df2['age'].mode()}, inplace = True)
```

- **Desventaja:** Puede reducir la varianza y afectar la correlación.



Técnicas para manejar valores faltantes III

- **Sustituir datos faltantes por un valor constante:** Útil cuando se puede hacer una suposición razonable sobre el valor.

```
# Imputación de datos  
df2.fillna({'age': 0}, inplace = True)
```

- **Desventaja:** Puede introducir sesgo si el valor elegido no es representativo.



Técnicas para manejar datos atípicos I

- **Eliminación:** Eliminar valores atípicos solo si están claramente mal registrados o medidos.

```
# Carga de librerías
from scipy import stats
import numpy as np

# Cálculo de puntajes Z, cuidado con los valores faltantes
z_scores = np.abs(stats.zscore(df2['fare']))
df2[z_scores <= 3]
```

- **Desventaja:** Puede causar pérdida de información si el valor no es un error.



Técnicas para manejar datos atípicos II

- **Transformación:** Las transformaciones pueden reducir el efecto de los valores extremos. Por ejemplo: Transformación logarítmica.

```
# Carga de librerías
import matplotlib.pyplot as plt
import numpy as np

# Transformación de datos
df['fare'] = np.log(df['fare'])

# Gráfico de dispersión
plt.scatter(range(len(df['fare'])), df['fare'])
plt.show()
```

- **Desventaja:** Puede alterar la interpretación de los datos.



Formato inconsistente

- Los datos categóricos suelen presentar variaciones en su escritura.
- Mayúsculas/minúsculas, espacios, abreviaciones, tipeo erróneo.
- Deben unificarse para que una misma categoría no sea tratada como varias.

```
# Estandarizar texto (minúsculas, sin espacios)
df['sex'] = df['sex'].str.lower().str.strip()

# Unificar categorías
df['embark_town'] = df['embark_town'].replace({'S': 'Southampton',
                                                'C': 'Cherbourg',
                                                'Q': 'Queenstown'})
```



Datos duplicados

- Registros idénticos (o casi idénticos) que aparecen más de una vez.

```
# Identificar filas duplicadas
duplicados = df.duplicated()
print("Filas duplicadas:", duplicados.sum())

# Mostrar duplicados (si existen)
df[duplicados]

# Eliminar duplicados (conservar el primero)
df.drop_duplicates(inplace=True)

# Eliminar duplicados considerando solo algunas columnas
df.drop_duplicates(subset=['pclass', 'sex', 'age'], keep='last',
inplace=True)
```



Datos redundantes

- Variables que expresan la misma información de forma diferente.

```
# Identificar redundancias mediante correlación o inspección
df[['survived', 'alive']].head()

# Eliminar variable redundante
df.drop('alive', axis=1, inplace=True)

# Otras redundancias comunes:
# - 'class' (First, Second, Third) y 'pclass' (1,2,3)
# - 'adult_male' derivado de 'age' y 'sex'
df.drop('class', axis=1, inplace=True)
```



Tarea 2: Ejercicio de limpieza de datos

- Utiliza el mismo conjunto de datos de la tarea anterior.
- Realiza las siguientes actividades de limpieza:
 - Crea una copia de trabajo del conjunto para no modificar el original.
 - Identifica valores faltantes y calcula su porcentaje por variable.
 - Maneja los valores faltantes y justifica tu decisión.
 - Detecta valores atípicos en tres variables numéricas con IQR o z-score.
 - Decide cómo tratar los valores atípicos y justifica.
 - Identifica y elimina registros duplicados si existen.
 - Identifica variables redundantes (correlación > 0.9 o valores repetidos).
 - Elimina las variables redundantes y justifica por qué no aportan valor.
 - Compara las dimensiones del conjunto antes y después de la limpieza.
- Entrega una libreta con el código, las transformaciones y tus justificaciones.



Transformación de datos

- Los datos transformados permiten que los algoritmos aprendan.
- Se aplican cambios sobre los datos limpios para:
 - Adaptar variables al formato requerido por el modelo.
 - Crear nuevas representaciones que capturen mejor la información.
 - Mejorar el rendimiento.
- Principales tipos de transformación:
 - Codificación de variables categóricas.
 - Escalado de variables numéricas.
 - Remuestreo para equilibrar clases.



Técnicas de codificación

- Las variables categóricas representan grupos o categorías.
- Algunos modelos de aprendizaje máquina requieren entradas numéricas.
- Transformar categorías en números para análisis y modelado.
- Técnicas comunes:
 - **One-Hot Encoding:** Para categorías sin orden (ej: puerto de embarque).
 - **Label Encoding:** Para categorías con orden (ej: nivel educativo).



One-hot encoding

- Crea una columna binaria (0/1) para cada categoría.

```
df = pd.get_dummies(df, columns=['embarked'], prefix='embarked')
```

- Columnas: embarked_S, embarked_C, embarked_Q.
- **Ventaja:** Elimina supuestos de orden entre categorías.



Label encoding

- Asigna un número único a cada categoría.

```
# Crear un diccionario de mapeo para 'sex'  
gender_map = {'male': 0, 'female': 1}  
df['sex_encoded'] = df['sex'].map(gender_map)
```

- **Ventaja:** Mantiene la dimensionalidad del conjunto de datos.



Desbalance de clases

- En problemas de clasificación, los datos desbalanceados ocurren cuando el número de observaciones en una clase es significativamente menor.
- Ejemplo:
 - Clase A: 90 muestras.
 - Clase B: 10 muestras.
- La mayoría de los algoritmos de clasificación están diseñados para maximizar la precisión y minimizar el error, provocando que el modelo se enfoque en la clase mayoritaria.
- La precisión puede parecer alta, pero la capacidad de predecir correctamente la clase minoritaria es baja.



Técnicas para manejar datos desbalanceados

- **Re-muestreo** (*Resampling*): Crear nuevas muestras a partir de existentes:
 - **Submuestreo** (*undersampling*): Reducir la clase mayoritaria.
 - **Sobremuestreo** (*oversampling*): Aumentar la clase minoritaria.



Submuestreo

- Elimina ejemplos de la clase mayoritaria para equilibrar las clases.

```
# Verificación de la distribución de clases
```

```
df['class'].value_counts()
```

```
# División del conjunto de datos
```

```
mayoritaria = df[df['class'] == 0]
```

```
minoritaria = df[df['class'] == 1]
```

```
# Selección aleatoria de la clase mayoritaria
```

```
submuestreo = mayoritaria.sample(len(minoritaria))
```

```
# Combinación de datos submuestreados
```

```
df_submuestreado = pd.concat([submuestreo, minoritaria])
```

```
# Comprobación
```

```
print(df_submuestreado ['class'].value_counts())
```



Sobremuestreo

- Duplica o genera nuevos ejemplos para la clase minoritaria.

```
# Verificación de la distribución de clases
df['class'].value_counts()

# Sobremuestreo manual
sobremuestreo = minoritaria.sample(len(mayoritaria), replace=True)

# Combinación de datos sobremuestreados
df_sobremuestreado = pd.concat([mayoritaria, sobremuestreo])

# Comprobación
print(df_sobremuestreado ['class'].value_counts())
```



Escalado de variables numéricas I

- Muchos algoritmos son sensibles a la escala de las variables.
- Variables con rangos diferentes pueden dominar el modelo.
- Técnicas comunes:
 - Normalización (Min-Max): Escala los valores a un rango $[0, 1]$.
 - Estandarización (Z-score): Centra en media 0 y desviación estándar 1.



Escalado de variables numéricas II

```
# Importación de funciones de escalado
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Normalización Min-Max
scaler_minmax = MinMaxScaler()
df['fare_minmax'] = scaler_minmax.fit_transform(df[['fare']])

# Estandarización Z-score
scaler_std = StandardScaler()
df['fare_std'] = scaler_std.fit_transform(df[['fare']])

# Comparar resultados
print(df[['fare', 'fare_minmax', 'fare_std']].head())
```



Tarea 3: Ejercicio de transformación de datos

- Trabaja con el conjunto de datos limpio de la tarea anterior.
- Realiza las siguientes actividades de transformación:
 - Identifica las variables categóricas nominales y ordinales en tu conjunto.
 - Aplica **One-Hot Encoding** a una variable nominal.
 - Aplica **Label Encoding** a una variable ordinal.
 - Muestra el conjunto antes y después de la codificación.
 - Selecciona dos variables numéricas para escalar.
 - Aplica normalización a una de ellas y estandarización a la otra.
 - Si la variable objetivo está desbalanceada, aplica remuestreo.
 - Verifica la nueva distribución de clases después del remuestreo.
 - Explica por qué elegiste cada técnica de transformación.
- Entrega una libreta con el código, las transformaciones y tus comentarios.



Aprendizaje supervisado

- Los humanos aprendemos de experiencias pasadas.
- Un niño aprende a identificar animales viendo ejemplos.
- La experiencia se acumula y permite generalizar a nuevos casos.
- Una computadora no tiene experiencias. En su lugar, aprende de datos.
- Estos datos son ejemplos históricos con entradas y salidas conocidas.
- El objetivo es obtener una función que prediga salidas para nuevas entradas.



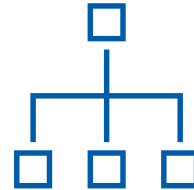
Flujo del aprendizaje supervisado

- Para generar un modelo se necesita un conjunto de datos, un algoritmo de aprendizaje y una forma de evaluarlo o interpretarlo.



Conjunto de datos

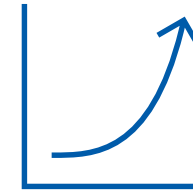
Variable 1	Variable 2	Objetivo
A	B	V_1
C	D	V_2
E	F	V_3



Modelo supervisado

Existen más de ciento cincuenta modelos clasificadores [9]:

1. Regresión lineal
2. Redes neuronales
3. Redes bayesianas
4. Árboles de decisión



Interpretación/Evaluación

Una vez obtenido el modelo, se puede:

1. Analizar el conocimiento o los patrones capturados.
2. Medir su desempeño en datos no observados antes.

[9] Fernández-Delgado, M., Cernadas, E., Barro, S., & Amorim, D. (2014). Do we Need Hundreds of Classifiers to Solve Real World Classification Problems?



Datos en el aprendizaje supervisado

- Un conjunto de datos está compuesto por:
 - Una lista de **registros** (observaciones o ejemplos).
 - Un conjunto de variables **predictoras** (características o atributos).
 - Las variables **cuantitativas** se conocen como continuas o numéricas.
 - Las variables **cualitativas** se conocen como discretas o categóricas.
 - Dentro de las categóricas existen las nominales y las ordinales.
 - Una variable **por predecir** (dependiente, objetivo o clase).
- Si la variable objetivo es categórica se vuelve un problema de **clasificación**.
- Si la variable objetivo es numérica se vuelve un problema de **regresión**.



Conjunto de datos: Diamonds

id	carat	cut	color	clarity	depth	table	price	x	y	z
0	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
1	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
2	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
3	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
4	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75
5	0.24	Very Good	J	VVS2	62.8	57.0	336	3.94	3.96	2.48
6	0.24	Very Good	I	VVS1	62.3	57.0	336	3.95	3.98	2.47
7	0.26	Very Good	H	SI1	61.9	55.0	337	4.07	4.11	2.53
8	0.22	Fair	E	VS2	65.1	61.0	337	3.87	3.78	2.49
9	0.23	Very Good	H	VS1	59.4	61.0	338	4.00	4.05	2.39



Características de Diamonds

- **Uso:** Regresión (price) o clasificación (cut).

Variable	Tipo	Descripción
price	numérica continua	Precio en dólares
carat	numérica continua	Peso del diamante
cut	categorica ordinal	Calidad del corte (Fair, Good, Very Good, Premium, Ideal)
color	categorica ordinal	Color (D=mejor a J=peor)
clarity	categorica ordinal	Claridad (I1, SI2, SI1, VS2, VS1, VVS2, VVS1, IF)
depth	numérica continua	Profundidad $2*z/(x+y)$
table	numérica continua	Ancho de la mesa
x	numérica continua	Largo en mm
y	numérica continua	Ancho en mm
z	numérica continua	Profundidad en mm



Modelos supervisados de aprendizaje

- Un modelo es una **función** o **teoría** que se aprende de los datos.
- Puede verse como una **hipótesis** sobre la relación entre entradas y salidas.
- Se **ajusta** (entrena) un modelo usando un algoritmo de aprendizaje.
- El algoritmo (método o técnica) encuentra los **parámetros** del modelo.
- Los algoritmos se configuran a través de sus **hiperparámetros**.
- Una vez entrenado, el modelo puede predecir salidas para nuevos datos.

- Si la variable objetivo es categórica, el modelo se llama **clasificador**.
- Si la variable objetivo es numérica, se llama modelo de regresión.



Interpretación y evaluación de modelos

- Los modelos no solo predicen, también **representan conocimiento**.
- Ese conocimiento revela **patrones** sobre cómo se relacionan las variables.
- Es importante para **tomar decisiones** en situaciones complejas.
- Algunos modelos permiten interpretar ese conocimiento.
- Los modelos que no se pueden interpretar se denominan de **caja negra**.
- Las redes neuronales son un ejemplo clásico de caja negra.
- **Evaluar** un modelo es tan importante como entrenarlo.
- La evaluación determina si el modelo es confiable o debe sustituirse.



Modelo de regresión lineal

- Describe la **relación** entre una variable independiente y una dependiente.
- Asume que dicha relación puede representarse mediante una **línea recta**.
- Permite **predecir** valores continuos a partir de un solo atributo.
- Es el punto de partida para entender las tareas de **regresión**.



Forma del modelo

$$\hat{y} = \theta_0 + \theta_1 x$$

Donde:

- $\hat{y}$: valor predicho por el modelo (estimación de la variable dependiente)
 - x : variable independiente
 - θ_0 : intercepto que representa el valor de $\hat{y}$ cuando $x = 0$
 - θ_1 : pendiente que representa el cambio en $\hat{y}$ por cada unidad en x
-
- El objetivo del modelo es que $\hat{y}$ sea lo más cercano posible a y .
 - El modelo define una familia de todas las líneas posibles.
 - Los parámetros $\theta = [\theta_0, \theta_1]^T$ determinan la línea específica.



Ejemplo sobre el salario mensual

- Se cuenta con datos de siete personas con diferentes años de experiencia y sus salarios mensuales en miles de pesos.

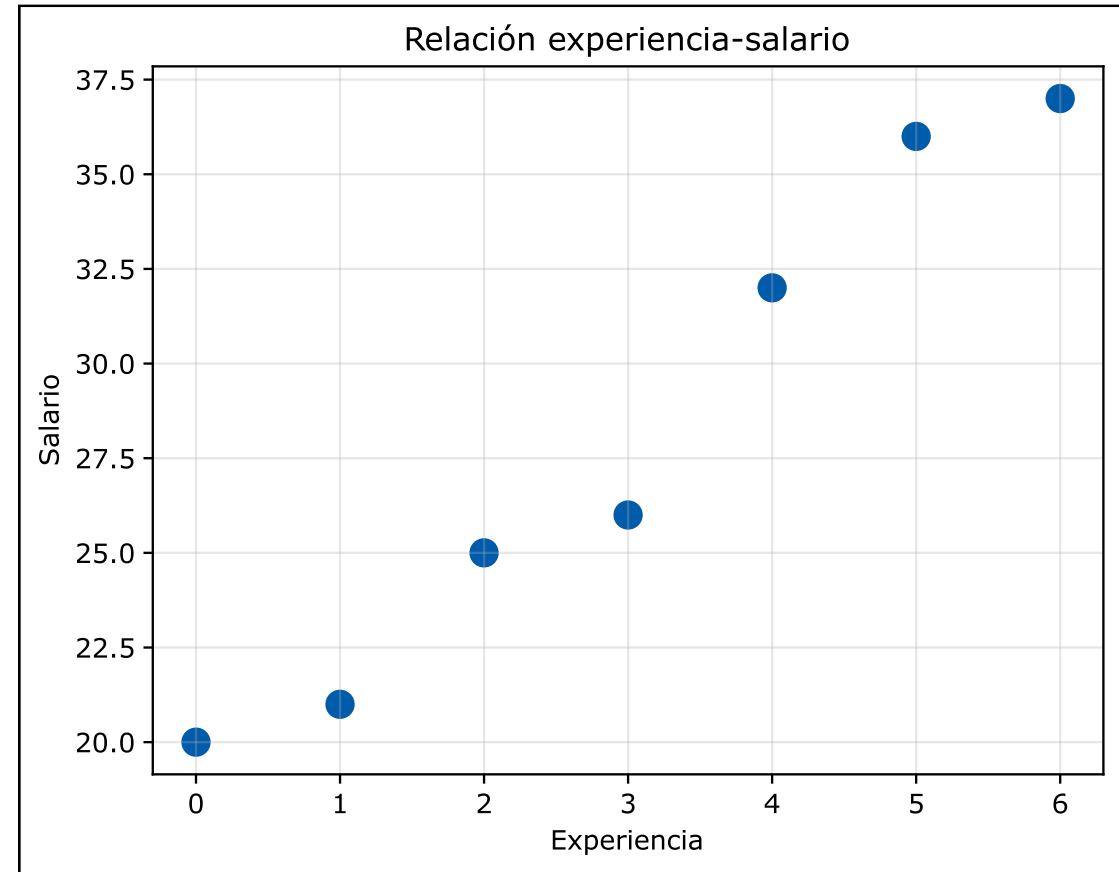
Experiencia	0	1	2	3	4	5	6
Salario	20	21	25	26	32	36	37



Visualización de la relación entre dos variables

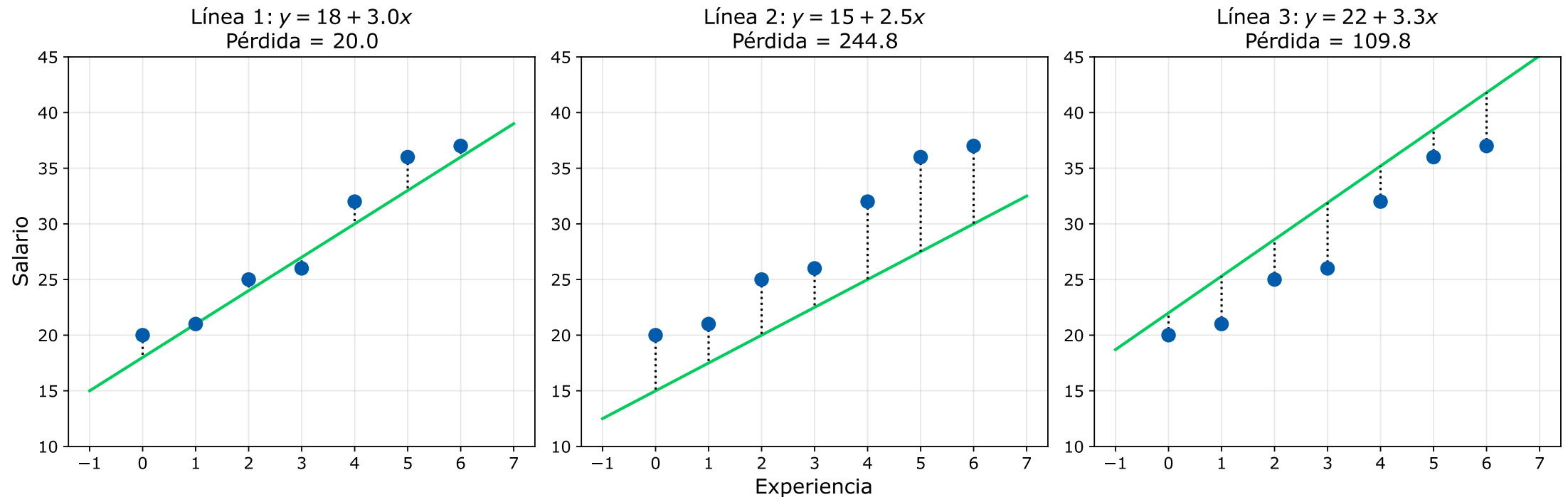
```
# Importar NumPy y Matplotlib
# Datos
x = np.array([0, 1, 2, 3, 4, 5, 6])
y = np.array([20, 21, 25, 26, 32, 36, 37])

# Gráfica de puntos
plt.scatter(x, y, color='#005BAA', s=100)
plt.xlabel('Experiencia')
plt.ylabel('Salario')
plt.title('Relación experiencia-salario')
plt.grid(True, alpha=0.3)
plt.show()
```



Ajuste de diferentes parámetros

- Cada elección de θ_0 y θ_1 define una línea diferente.
- Algunas líneas se ajustan mejor a los datos que otras.
- La calidad del ajuste se mide mediante la función de pérdida.



Visualización de múltiples parámetros

```
lineas = [(18, 3.0, 20), (15, 2.5, 244.8), (22, 3.3, 109.8)]
x_plot = np.linspace(-1, 7, 100)
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

for i, (t0, t1, p) in enumerate(lineas):
    y_pred = t0 + t1 * x
    axes[i].scatter(x, y, color='#005BAA', s=80, zorder=3)
    axes[i].plot(x_plot, t0 + t1 * x_plot, '-', color="#00CC5E", linewidth=2)

    for xi, yi, ypi in zip(x, y, y_pred):
        axes[i].plot([xi, xi], [yi, ypi], color='black', linestyle=':')

    # Configurar título, etiquetas, ejes y rejilla
    axes[i].set_title(f'Línea {i+1}:  $y = \{t0\} + \{t1\}x$ \nPérdida =  $\{p:.1f\}$ ', fontsize=14)
    axes[i].tick_params(axis='both', which='major', labelsize=11)
    axes[i].grid(True, alpha=0.3)
    axes[i].set_ylim(10, 45)

axes[1].set_xlabel('Experiencia', fontsize=14)
axes[0].set_ylabel('Salario', fontsize=14)

plt.tight_layout()
plt.show()
```

Parámetros y pérdidas
Valores para las líneas
Figura con 3 columnas

Bucle sobre las líneas

Dibujar puntos
Dibujar líneas

Dibujar residuos



Función de pérdida (*Loss Function*)

- Mide qué tan mal se ajusta el modelo a los datos.
- Para cada punto, calculamos el error: diferencia entre valor real y predicho.
- Se usa el error cuadrático para que la dirección no importe.
- La pérdida total es la suma de errores cuadráticos (SSE).

$$L[\theta_0, \theta_1] = \sum_i (y_i - \hat{y}_i)^2$$

$$L[\theta_0, \theta_1] = \sum_i (y_i - \theta_0 - \theta_1 x_i)^2$$



Ejercicio: cálculo de la suma del error cuadrático

- A partir del conjunto de datos sobre salarios, predice las salidas de los siguientes modelos y luego calcula la suma de errores cuadrados:

Experiencia	0	1	2	3	4	5	6
Salario	20	21	25	26	32	36	37

Modelo	θ_0	θ_1
1	13	5
2	18	-3



Programación de la función de pérdida

```
def funcion_perdida(x, y, theta0, theta1):  
    """  
    Calcula la función de pérdida de mínimos cuadrados (SSE).  
    Parámetros:  
    x : Vector de valores de la variable independiente.  
    y : Vector de valores reales de la variable dependiente.  
    theta0 : Intercepto del modelo de regresión lineal.  
    theta1 : Pendiente del modelo de regresión lineal.  
  
    Retorna:  
    Suma de cuadrados de los errores (SSE).  
    """  
    predicciones = theta0 + theta1 * x  
    errores = y - predicciones  
    return np.sum(errores ** 2)
```



Ajuste de parámetros en forma cerrada

- El método de mínimos cuadrados busca los parámetros óptimos θ_0 y θ_1 .
- Se minimiza la suma de errores cuadráticos: $\sum_i (\theta_0 + \theta_1 x_i - y_i)^2$.
- Para la regresión lineal simple, existe una solución analítica cerrada.
- Se obtiene igualando las derivadas parciales de la función de pérdida a cero.
- La pendiente e intercepto óptimos se calculan de la siguiente manera:

$$\theta_1 = \frac{\text{Cov}(x, y)}{\text{Var}(x)} = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$$

$$\theta_0 = \bar{y} - \theta_1 \bar{x}$$

- Donde $\bar{x}$ y $\bar{y}$ son las medias muestrales de x y y , respectivamente.



Ejercicio: cálculo de parámetros en forma cerrada

- A partir del conjunto de datos sobre salarios, calcula las medias de cada variable, la varianza del atributo y la covarianza. Después, calcula θ_1 y θ_0 .

Experiencia	0	1	2	3	4	5	6
Salario	20	21	25	26	32	36	37



Ejercicio: cálculo de parámetros en forma cerrada

- Carga el conjunto de datos `penguins` directamente de `seaborn`.
- Obtén la matriz de correlación usando el método `corr()` del dataframe.
- Selecciona un par de variables con un coeficiente superior a 0.7.
- Calcula los parámetros θ_0 y θ_1 en forma cerrada.

```
# Calcular medias
x_media = np.mean(x)
y_media = np.mean(y)
# Calcular pendiente
numerador = np.sum((x - x_media) * (y - y_media))
denominador = np.sum((x - x_media) ** 2)
theta1_opt = numerador / denominador
# Calcular intercepto
theta0_opt = y_media - theta1_opt * x_media
```



Regresión Lineal con scikit-learn

- No es necesario implementar las fórmulas manualmente.
- Requiere que X sea una matriz de 2 dimensiones (incluso para una variable).

```
# Calcular medias
from sklearn.linear_model import LinearRegression

# Preparar datos
X = x.reshape(-1, 1)  # Convertir a matriz

# Crear y entrenar modelo
rl = LinearRegression()
rl.fit(X, y)

# Obtener parámetros
print(f"Theta0: {rl.intercept_:.2f}, theta1: {rl.coef_[0]:.2f}")
```



Predicción con scikit-learn

- Los datos de prueba y entrenamiento deben tener la misma forma.
- `predict()` aplica el modelo a los nuevos datos y devuelve las predicciones.
- `zip()` empareja cada ejemplo de prueba con su predicción para mostrarlas.

```
X_prueba = np.array([2.5, 5.5, 8.5]).reshape(-1, 1) # Experiencia
predicciones = rl.predict(X_prueba)                 # Predeicciones

for experiencia, salario in zip(X_prueba, predicciones):
    print(f"Experiencia {experiencia[0]}, predicción {salario:.2f}")
```



Evaluación de los modelos de predicción

- Entrenar un modelo no es suficiente; se debe de evaluar su rendimiento.
- Las métricas cuantifican la calidad de las predicciones.
- Permiten comparar diferentes modelos y seleccionar el mejor.
- Guían el ajuste de hiperparámetros y la toma de decisiones.
- Una buena métrica debe ser interpretable y relevante para el problema.



Métricas de error más comunes

Métrica	Características	Fórmula
MSE (<i>Mean Squared Error</i>) Promedio de errores al cuadrado	Penaliza errores grandes. Unidades al cuadrado.	$\text{MSE} = \frac{\sum_i (y_i - \hat{y}_i)^2}{n}$
RMSE (<i>Root Mean Squared Error</i>) Raíz cuadrada del MSE	Interpretación directa. Unidades originales.	$\text{RMSE} = \sqrt{\frac{\sum_i (y_i - \hat{y}_i)^2}{n}}$
MAE (<i>Mean Absolute Error</i>) Promedio de errores absolutos	Robusto a valores atípicos. Unidades originales.	$\text{MAE} = \frac{\sum_i y_i - \hat{y}_i }{n}$
R² (<i>Determination Coefficient</i>) Coeficiente de determinación	Proporción de varianza explicada. Rango [0,1]; 1 es ajuste perfecto.	$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$



Evaluación usando scikit-learn

- Una vez entrenado el modelo, se generan predicciones sobre la prueba.
- scikit-learn proporciona funciones para calcular métricas de evaluación.
- Todas las métricas comparan los valores reales y valores predichos.
- El orden de los argumentos es siempre (y_real, y_pred).

```
from sklearn.metrics import mean_squared_error, mean_absolute_error
from sklearn.metrics import r2_score
```

```
y_pred = rl.predict(X)
```

```
print(f"MSE    = {mean_squared_error(y, y_pred):.2f}")
print(f"RMSE   = {np.sqrt(mean_squared_error(y, y_pred)):.2f}")
print(f"MAE    = {mean_absolute_error(y, y_pred):.2f}")
print(f"R²     = {r2_score(y, y_pred):.4f}")
```



División del conjunto de datos

- Entrenar y evaluar con los mismos datos da una visión optimista.
- El modelo podría simplemente memorizar (sobreajuste) en vez de aprender.
- Se necesita simular cómo se comportará con datos nuevos no vistos.
- La división permite estimar la capacidad de generalización del modelo.



Conjuntos de entrenamiento y prueba

- Entrenamiento (**training set**): datos con los que el modelo aprende.
- Prueba (**test set**): datos que el modelo no ve durante el entrenamiento.
- Se usan solo al final para evaluar el rendimiento real.
- Proporción típica: 70-80% entrenamiento, 20-30% prueba.

```
from sklearn.model_selection import train_test_split

# Dividir datos: 80% entrenamiento, 20% prueba
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f"Entrenamiento: {len(X_train)} ejemplos")
print(f"Prueba: {len(X_test)} ejemplos")
```



Tarea 4: Ajuste de un modelo de regresión lineal simple

- Utiliza un conjunto de datos precargado en [seaborn](#).
- Obtén la matriz de correlación usando el método `corr()` del dataframe.
- Selecciona un par de variables con un coeficiente superior a 0.7.
- Divide el conjunto de datos usando una proporción de 30% para prueba.
- Ajusta un modelo de regresión lineal con el 70% de los datos.
- Calcula las cuatro métricas de error sugeridas.



